LA CITÉ

Les Déclencheurs: Triggers

Plan

- **Ce cours a pour but de vous apprendre à :**

- Connaitre ce qu'est un **déclencheur (trigger)** ;
- Comprendre la **différence** entre **BEFORE** et **AFTER** ;
- Créer, configurer, tester et supprimer des **déclencheurs** ;
- Comprendre le rôle des **tables virtuelles (inserted, deleted)** ;
- Appliquer tout ça avec des **exemples concrets**

Les déclencheurs en SQL

- **Définition et Utilité des déclencheurs**
- Un **déclencheur (trigger)** est un **programme stocké dans la base de données** qui s'exécute **automatiquement** quand un **événement** se produit sur une table.
- **Exemple d'événement :**

✓ **INSERT** : on ajoute une ligne

✓ **UPDATE** : on modifie une ligne

✓ **DELETE** : on supprime une ligne

➢ Le déclencheur agit **sans qu'on l'appelle directement** : il est **déclenché automatiquement** par le SGBD

Les déclencheurs en SQL

- **Objectifs de l utilisation des déclencheurs:**

✓ **Automatiser des actions** (mise à jour d'un journal, d'une date, etc.)

✓ **Contrôler la cohérence des données**

✓ **Empêcher certaines actions** (ex : suppression interdite)

✓ **Synchroniser des tables**

- **Attention :**

➢ Les déclencheurs doivent être utilisés avec **prudence**, car ils peuvent :

✓ rendre le code plus complexe à déboguer ;

✓ ralentir certaines opérations si mal conçus.

Les déclencheurs en SQL

- **Objectifs de l utilisation des déclencheurs:**

✓ **Automatiser des actions** (mise à jour d'un journal, d'une date, etc.)

- Les déclencheurs permettent d'exécuter automatiquement certaines actions après une modification dans la base.
  Exemples :
- **Mettre à jour automatiquement** une **date de dernière modification** lorsqu'un enregistrement change.
- Enregistrer une opération dans un **journal (log)** pour garder une trace des insertions, suppressions ou mises à jour.

➢ Cela évite de devoir écrire manuellement ces opérations dans chaque requête.

Les déclencheurs en SQL

- **Objectifs de l utilisation des déclencheurs:**

✓ **Contrôler la cohérence des données**

- Les déclencheurs peuvent vérifier que les données insérées ou modifiées respectent des **règles de cohérence et d'intégrité.**

  Exemples :

- Interdire qu'un **salaire soit négatif**.
- Vérifier qu'une **quantité en stock ne devienne pas inférieure à zéro.**

➢ Le but est de **garantir la validité des données** avant qu'elles soient enregistrées.

Les déclencheurs en SQL

- **Objectifs de l utilisation des déclencheurs:**

✓ **Empêcher certaines actions** (ex : suppression interdite)

- Les déclencheurs **peuvent bloquer certaines opérations non désirées.**

  Exemples :

- **Empêcher la suppression d'un enregistrement important** (par exemple, d'un client encore associé à des commandes).
- Refuser la **modification d'un champ** particulier, comme une clé primaire.

➢ Cela permet de **protéger les données sensibles ou critiques**.

Les déclencheurs en SQL

- **Objectifs de l utilisation des déclencheurs:**

✓ **Synchroniser des tables**

- Les déclencheurs permettent de **maintenir la cohérence et synchroniser entre plusieurs tables dépendantes.**

Exemples :

- Lorsqu'on insère une commande, mettre à jour automatiquement le **total du client** dans une autre table.
- Lorsqu'on supprime un enregistrement, supprimer automatiquement les **éléments liés** dans une autre table.

➢ L'objectif est de **garder les tables cohérentes entre elles**.

Les déclencheurs en SQL

- **Les différents types de déclencheurs selon le moment d'exécution**

✓ **BEFORE: n est pas supporté par SQL server**

- Le déclencheur s'exécute **avant l'action** (INSERT, UPDATE ou DELETE). Ça sert à :
- **Vérifier** les valeurs **avant** l'insertion ou la mise à jour
- **Modifier** les valeurs **avant qu'elles soient enregistrées**
- **Empêcher** une action (**en levant une erreur**)
- **Exemple :**

  Avant une mise à jour, vérifier qu'un salaire n'est pas négatif.

Les déclencheurs en SQL

- **Les différents types de déclencheurs selon le moment d'exécution**

✓ **AFTER**

- Le déclencheur s'exécute **aprés l'action** (INSERT, UPDATE ou DELETE). Ça sert à :
- **Mettre à jour d'autres tables** après une modification
- **Enregistrer des logs** ou **des historiques**
- **Synchroniser** des **tables**
- **Exemple :**

Après l'ajout d'une commande, **mettre à jour** le **total du client**.

Les déclencheurs en SQL

- **Les différents types de déclencheurs selon le moment d'exécution**

✓ **INSTEAD OF**

- Le déclencheur s'exécute **à la place** de l'action demandée. Autrement dit, l'action (INSERT, UPDATE, DELETE) ne se fait **pas directement**, **c'est le déclencheur qui la remplace**. Ça sert à :
- **Contrôler complètement** ce qui se passe quand quelqu'un **essaie** d'insérer, modifier ou supprimer
- **Exemple :**
  Sur une Table Employé, on peut créer un **INSTEAD OF INSERT** qui va empêcher l'insertion d'un employé dont le salaire est inférieur à 2000 et à la place de l insertion afficher un message d erreur comme insertion refusé et accepter les autres insertions des salaires supérieur à 2000 normalement.

Les déclencheurs en SQL

- **Les tables virtuelles inserted et deleted dans SQL SERVER:**
- Lorsque l'on utilise des **déclencheurs (triggers)** dans SQL Server, le système met automatiquement à disposition deux **tables virtuelles** : **inserted** et **deleted.**
- Ces tables ne sont **pas des tables physiques**, mais des structures **temporaires** accessibles uniquement **pendant l'exécution du déclencheur**.
- Elles permettent au trigger de savoir :
- quelles **lignes ont été ajoutées**
- quelles **lignes ont été modifiées**
- quelles **lignes ont été supprimées**

➢ Elles sont essentielles pour **contrôler, valider ou enregistrer** les **changements effectués sur une table**.

Les déclencheurs en SQL

- **Table virtuelle inserted**
- La table inserted contient les **nouvelles lignes** :
- celles qui viennent d'être **insérées** dans la table cible (avec INSERT),
- ou celles qui ont été **modifiées** (dans une opération UPDATE).

➢ **inserted** = **état après l'opération**.

- **Table virtuelle deleted**
- La table deleted contient les **anciennes lignes** :
- celles qui viennent d'être **supprimées** (DELETE),
- ou celles qui ont été **modifiées** (UPDATE).

➢ **deleted** = **état avant l'opération**.

✓ Elles servent à comparer l'ancien et le nouveau contenu.

Les déclencheurs en SQL

- Syntaxe générale d un déclencheur en SQL SERVER

```
CREATE TRIGGER nom_du_declencheur
ON nom_de_table
AFTER | INSTEAD OF | BEFORE  -- Moment de l exécution de l évènement
(attention : BEFORE n'existe pas dans SQL Server)
INSERT| UPDATE| DELETE –type de l évènement
AS
BEGIN
   -- Instructions  à exécuter quand l'événement se produit
END
```

Les déclencheurs en SQL

• Syntaxe générale d un déclencheur en SQL SERVER

On va travailler directement avec un exemple concret

- **Créer la base gestionclients avec la table principale Clients et la Table Journal pour enregistrer les actions déclenchées**
- **Utiliser le fichier base gestionclients.sql**

Exercices procédu...LCCAD\WOuesl (53)) base gestionclients.sql - non conn

```sql
CREATE DATABASE GestionClients;

-- Utilisation de la base
USE GestionClients;

-- Table principale
CREATE TABLE Clients (
    Id_Client INT IDENTITY(1,1) PRIMARY KEY,
    Nom NVARCHAR(100),
    Pays NVARCHAR(50)
);

-- Table pour journaliser les actions
CREATE TABLE Journal (
    Id_Journal INT IDENTITY(1,1) PRIMARY KEY,
    Action NVARCHAR(200),
    DateAction DATETIME
);
```

100 %

Messages

Commandes réussies.

Heure de fin : 2025-11-02T12:02:07.2377079-05:00

Les déclencheurs en SQL

• Syntaxe générale d un déclencheur en SQL SERVER

On va travailler directement avec un exemple concret

- **Déclencheurs AFTER: AFTER INSERT**
- **Exemple**

On veut créer un déclencheur qui met des informations sur le client inseré dans la table client (son nom et la date de l insertion) dans la table journal

```sql
-- déclencheur aprés chaque insertion dans la table client pour alimenter la table journal par le non du client inseré et la date de l insertion
CREATE TRIGGER trg_AfterInsert_Client
ON Clients
AFTER INSERT--Ce déclencheur s'exécute après une insertion réussie dans Clients.
AS
BEGIN
    INSERT INTO Journal (Action, DateAction)
    SELECT
        'Nouveau client ajouté : ' + Nom,
        GETDATE()--donne la date de l action
    FROM inserted;--La table virtuelle inserted contient les nouvelles lignes ajoutées dans Clients.
END;
```

100 %

Messages

```
Commandes réussies.

Heure de fin : 2025-11-03T10:28:41.9760047-05:00
```

Les déclencheurs en SQL

• Syntaxe générale d un déclencheur en SQL SERVER

On va travailler directement avec un exemple concret

- **Déclencheurs AFTER: AFTER INSERT**
- **Exemple**

Effectuer une insertion dans la table Clients puis afficher le contenu de la table journal pour voir le travail effectué par le déclencheur automatiquement après l insertion du client :AFTER INSERT

```sql
--test after insert dans la table Clients
INSERT INTO Clients (Nom, Pays) VALUES ('Alice', 'Canada');
--verifier que les informations ont été enregistré dans la tabl
SELECT * FROM Journal;
```

100 %

Messages

```
(1 ligne affectée)

(1 ligne affectée)

Heure de fin : 2025-11-03T10:31:53.4776032-05:00
```

```sql
--verifier que les informations ont été enregistré dans la table journal
SELECT * FROM Journal;
```

00 %

Résultats | Messages

	Id_Journal	Action	DateAction
1	1	Nouveau client ajouté : Alice	2025-11-03 10:31:53.467

Les déclencheurs en SQL

- **Syntaxe générale d un déclencheur en SQL SERVER**

On va travailler directement avec un exemple concret

- **Déclencheurs AFTER: AFTER UPDATE**
- **Exemple**
- On veut créer un déclencheur qui met des informations sur le client mis à jour dans la table clients dans la table journal. Exemple : Client mis à jour : Ancien pays = Canada , Nouveau pays = France, 2025-11-03 10:57:24.980

```sql
-- déclencheur aprés chaque mise à jour dans la table client pour alimenter la table journal par le type de mise à jour  effectué dans clients et la date
CREATE TRIGGER trg_AfterUpdate_Client
ON Clients
AFTER UPDATE
AS
BEGIN
    INSERT INTO Journal (Action, DateAction)
    SELECT
        'Client mis à jour : Ancien pays = ' + d.Pays + ', Nouveau pays = ' + i.Pays,
        GETDATE()
    FROM deleted d --deleted est une table virtuelle qui contient les anciennes lignes avant la mise à jour,
    INNER JOIN inserted i ON d.Id_Client = i.Id_Client;--inserted est une table virtuelle qui contient les nouvelles lignes aprés la mise à jour,
END;
```

100 %

Messages

```
Commandes réussies.

Heure de fin : 2025-11-03T10:55:00.9429311-05:00
```

Les déclencheurs en SQL

• Syntaxe générale d un déclencheur en SQL SERVER

On va travailler directement avec un exemple concret

- **Déclencheurs AFTER: AFTER UPDATE**
- **Exemple**

Effectuer une mise à jour dans la table Clients puis afficher le contenu de la table journal pour voir le travail effectué par le déclencheur automatiquement après la mise à jour du client :AFTER UPDATE

```
--test after update dans la table Clients
UPDATE Clients SET Pays = 'France' WHERE Nom = 'Alice';
```

100 %

Messages

```
(1 ligne affectée)

(1 ligne affectée)

Heure de fin : 2025-11-03T10:57:24.9929510-05:00
```

```
--verifier que les informations de mise à jour du client ont été enregistré dans la table journal
SELECT * FROM Journal;
```

100 %

Résultats | Messages

	Id_Journal	Action	DateAction
1	1	Nouveau client ajouté : Alice	2025-11-03 10:31:53.467
2	2	Client mis à jour : Ancien pays = Canada, Nouve...	2025-11-03 10:57:24.980

Les déclencheurs en SQL

- **Syntaxe générale d un déclencheur en SQL SERVER**

On va travailler directement avec un exemple concret

- **Déclencheurs AFTER: AFTER DELETE**
- **Exemple**
- On veut créer un déclencheur qui met des informations sur le client supprimé de la table clients dans la table journal. Exemple :Client supprimé : Alice, 2025-11-03 12:13:00.760

```sql
-- déclencheur aprés chaque suppression dans la table client pour alimenter la table journal par les informations supprimées
CREATE TRIGGER trg_AfterDelete_Client
ON Clients
AFTER DELETE --Ce déclencheur agit après suppression d'un client.
AS
BEGIN
    INSERT INTO Journal (Action, DateAction) --insertion du nom du client supprimé et de la date de suppression dans la table
    SELECT
        'Client supprimé : ' + Nom,
        GETDATE()
    FROM deleted; --table virtuelle qui contient les contient les lignes supprimées
END;
```

100 %

Messages

```
Commandes réussies.

Heure de fin : 2025-11-03T12:08:30.2566938-05:00
```

Les déclencheurs en SQL

• Syntaxe générale d un déclencheur en SQL SERVER

On va travailler directement avec un exemple concret

- **Déclencheurs AFTER: AFTER DELETE**
- **Exemple**

Effectuer une suppression d un client de la table Clients puis afficher le contenu de la table journal pour voir le travail effectué par le déclencheur automatiquement après la suppression du client :AFTER DELETE

```
--test after delete dans la table Clients par la suppression d un client
DELETE FROM Clients WHERE Nom = 'Alice';
```

100 %

Messages

```
(1 ligne affectée)

(1 ligne affectée)

Heure de fin : 2025-11-03T12:13:00.7757052-05:00
```

```
--verifier que les informations de suppression du client ont été enregistré dans la table journal
SELECT * FROM Journal;
```

100 %

Résultats | Messages

	Id_Journal	Action	DateAction
1	1	Nouveau client ajouté : Alice	2025-11-03 10:31:53.467
2	2	Client mis à jour : Ancien pays = Canada, Nouve...	2025-11-03 10:57:24.980
3	3	Client supprimé : Alice	2025-11-03 12:13:00.760

Les déclencheurs en SQL

- **Syntaxe générale d un déclencheur en SQL SERVER**
- **Déclencheurs AFTER: AFTER INSERT, UPDATE**
- **Exemple On veut créer un déclencheur qui met des informations sur le client inseré ou le client mis à jour dans la table clients dans la table journal.**

base gestionclient...LCCAD\WOuesl (63))* | Exercices procédure....sql - non connecté | Exercices procédure....sql - non connecté* | Exercices procédure....sql - non connecté

```sql
--declencheur hybride after insert un nouveau client ou  update d un ancien client
CREATE TRIGGER trg_AfterInsertUpdateClient
ON Clients
AFTER INSERT, UPDATE
AS
BEGIN
    -- Pour insertion
    INSERT INTO Journal (Action, DateAction)
    SELECT
        'Nouveau client ajouté : ' + i.Nom + ' (' + i.Pays + ')', GETDATE()
    FROM inserted i
    LEFT JOIN deleted d ON i.Id_Client = d.Id_Client-- On fait une jointure externe gauche (LEFT JOIN) entre inserted et deleted.

                                                    --Si la clé (Id_Client) n'existe pas dans deleted, c'est une nouvelle ligne → donc un INSERT.
    WHERE d.Id_Client IS NULL;  -- Présent uniquement dans inserted et non pas dans deleted → INSERT

    -- Pour mise à jour
    INSERT INTO Journal (Action, DateAction)
    SELECT
        'Client modifié : ' + i.Nom +
        ' (Pays : ' + d.Pays + ' ' + i.Pays + ')',
        GETDATE()
    FROM inserted i
    INNER JOIN deleted d ON i.Id_Client = d.Id_Client; --  Quand une ligne est mise à jour, elle existe dans les deux (inserted et deleted).

    PRINT 'Action(s) journalisée(s) avec succès.';
END;
```

Les déclencheurs en SQL

• Syntaxe générale d un déclencheur en SQL SERVER

On va travailler directement avec un exemple concret

- **Déclencheurs hybride AFTER: AFTER INSERT, UPDATE**
- **Exemple**

Effectuer une insertion d un nouveau client de la table Clients

```
END;
--verification de l insertion
INSERT INTO Clients (Nom, Pays)
VALUES ('Alice', 'France');
--verification de la mise à jour du pays du client qui s appelle Alice
UPDATE Clients
SET Pays = 'Canada'
WHERE Nom = 'Alice';
```

100 %

Messages

```
(1 ligne affectée)

(1 ligne affectée)

(0 lignes affectées)
Action(s) journalisée(s) avec succès.

(1 ligne affectée)

Heure de fin : 2025-11-08T08:58:09.3036775-05:00
```

Les déclencheurs en SQL

• Syntaxe générale d un déclencheur en SQL SERVER

On va travailler directement avec un exemple concret

- **Déclencheurs hybride AFTER: AFTER INSERT, UPDATE**
- **Exemple**

Effectuer une mise à jour dans la table Clients en changeant le pays du client qui s appelle Alice

```sql
--verification de la mise à jour du pays du client qui s appelle Alice
UPDATE Clients
SET Pays = 'Canada'
WHERE Nom = 'Alice';
```

100 %

Messages

```
(1 ligne affectée)

(0 lignes affectées)

(1 ligne affectée)
Action(s) journalisée(s) avec succès.

(1 ligne affectée)

Heure de fin : 2025-11-08T09:00:46.5264869-05:00
```

Les déclencheurs en SQL

• Syntaxe générale d un déclencheur en SQL SERVER

On va travailler directement avec un exemple concret

- **Déclencheurs hybride AFTER: AFTER INSERT, UPDATE**
- **Exemple**

Afficher le contenu de la table journal pour voir le travail effectué par le déclencheur automatiquement après l insertion du client et la mise à jour :AFTER INSERT,update

```
WHERE Nom = 'Alice';
--verification de la journalisation de l insertion et de la mise à jour
SELECT * FROM Journal
```

100 %

Résultats | Messages

	Id_Journal	Action	DateAction
1	1	Nouveau client ajouté : Alice	2025-11-03 10:31:53.467
2	2	Client mis à jour : Ancien pays = Canada, Nouveau ...	2025-11-03 10:57:24.980
3	3	Client supprimé : Alice	2025-11-03 12:13:00.760
4	4	Nouveau client ajouté : Bob	2025-11-03 12:47:50.970
5	5	Client mis à jour : Ancien pays = Maroc, Nouveau p...	2025-11-03 13:24:50.923
6	6	Mise à jour via la vue : Nouveau Nom = Wided, No...	2025-11-03 13:24:50.923
7	7	Client supprimé : Wided	2025-11-03 13:54:52.510
8	8	Suppression via la vue : Wided	2025-11-03 13:54:52.510
9	9	Nouveau client ajouté : Alice	2025-11-08 08:58:09.287
10	10	Nouveau client ajouté : Alice (France)	2025-11-08 08:58:09.293
11	11	Client mis à jour : Ancien pays = France, Nouveau ...	2025-11-08 09:00:46.510
12	12	Client modifié : Alice (Pays : France ? Canada)	2025-11-08 09:00:46.513

Les déclencheurs en SQL

• Syntaxe générale d un déclencheur en SQL SERVER

On va travailler directement avec un exemple concret

- **Déclencheurs INSTEAD OF:**

Les déclencheurs INSTEAD OF s'utilisent souvent sur une vue, car ils remplacent l'action par un comportement personnalisé.

✓ **Créons une vue VueClients**

CREATE VIEW VueClients AS

SELECT Id_Client, Nom, Pays FROM Clients;

```
--creation d une vue pour utiliser les declencheurs INSTEAD OF
CREATE VIEW VueClients AS
SELECT Id_Client, Nom, Pays FROM Clients;
```

100 %

Messages

```
Commandes réussies.

Heure de fin : 2025-11-03T12:24:19.3114147-05:00
```

Les déclencheurs en SQL

• Syntaxe générale d un déclencheur en SQL SERVER

On va travailler directement avec un exemple concret

- **Déclencheurs INSTEAD OF: INSTEAD OF INSERT**
- **Exemple**
- On veut créer un déclencheur qui Remplace une tentative d'insertion d un client sur la vue par une insertion réelle dans la table Clients et l enregistrement de cette action dans la table journal

```sql
--   déclencheur aprés chaque tentative d insertion dans la vue Vueclients qui va inserer ces informations
--dans la table client au lieu de les mettres dans la vue et enregistrer ces informations dans la table journal
CREATE TRIGGER trg_InsteadOfInsert_VueClients
ON VueClients
INSTEAD OF INSERT-- au lieu d inserer dans la Vueclients faites 1 et 2
AS
BEGIN
    INSERT INTO Clients (Nom, Pays)-- 1:inserer dans la table client
    SELECT Nom, Pays FROM inserted;

    INSERT INTO Journal (Action, DateAction) --2: inserer les informations de l insertion dans la table journal
    SELECT
        'Insertion via la vue : ' + Nom,
        GETDATE()
    FROM inserted;
END;
```

100 %

Messages

```
Commandes réussies.

Heure de fin : 2025-11-03T12:24:19.3114147-05:00
```

Les déclencheurs en SQL

• Syntaxe générale d un déclencheur en SQL SERVER

On va travailler directement avec un exemple concret

- **Déclencheurs INSTEAD OF: INSTEAD OF INSERT**
- **Exemple**

Effectuer une INSERTION dans la vue Vueclients puis afficher le contenu de la table clients et journal pour voir le travail effectué par le déclencheur automatiquement après l insertion dans la vue

```
-- inserer un client dans la vue pour declencher automatiquement le déclencheur trg_InsteadOfInsert_VueClients
INSERT INTO VueClients (Nom, Pays) VALUES ('Bob', 'Maroc');
```

100 %

Messages

```
(1 ligne affectée)

(1 ligne affectée)

Heure de fin : 2025-11-03T12:47:50.9828373-05:00
```

Les déclencheurs en SQL

• Syntaxe générale d un déclencheur en SQL SERVER

On va travailler directement avec un exemple concret

- **Déclencheurs INSTEAD OF: INSTEAD OF INSERT**
- **Exemple**

Effectuer une INSERTION dans la vue Vueclients puis afficher le contenu de la table clients et journal pour voir le travail effectué par le déclencheur automatiquement après l insertion dans la vue

```
--dans la table clients au lieu de la vue grace au declencheur trg_InsteadOfInsert_VueClients
SELECT * FROM Clients;
```

100 %

Résultats | Messages

	Id_Client	Nom	Pays
1	2	Bob	Maroc

l insertion à été faite dans la table Clients

INSTEAD OF INSERT dans la vue VueClients

```
--verifier que les informations de l insertion du client dans la table client
--au lieu de la vue ont  été enregistré dans la table journal
SELECT * FROM Journal;
```

100 %

Résultats | Messages

	Id_Journal	Action	DateAction
1	1	Nouveau client ajouté : Alice	2025-11-03 10:31:53.467
2	2	Client mis à jour : Ancien pays = Canada, Nouve...	2025-11-03 10:57:24.980
3	3	Client supprimé : Alice	2025-11-03 12:13:00.760
4	4	Nouveau client ajouté : Bob	2025-11-03 12:47:50.970

Les déclencheurs en SQL

- **Syntaxe générale d un déclencheur en SQL SERVER**

On va travailler directement avec un exemple concret

- **Déclencheurs INSTEAD OF: INSTEAD OF UPDATE**
- **Exemple**
- On veut créer un déclencheur qui Remplace une tentative de mise à jour d un client sur la vue par une mise à jour réelle dans la table Clients et l enregistrement de cette action dans la table journal

```sql
-- creation de déclencheur qui remplace la mise à jour dans la vue par une mise à jour réélle
--dans la table client avec une journalisation de la table journal
CREATE TRIGGER trg_InsteadOfUpdate_VueClients
ON VueClients
INSTEAD OF UPDATE
AS
BEGIN
    UPDATE c
    SET
        c.Nom = i.Nom,
        c.Pays = i.Pays
    FROM Clients c
    INNER JOIN inserted i ON c.Id_Client = i.Id_Client;

    INSERT INTO Journal (Action, DateAction)
    SELECT 'Mise à jour via la vue : Nouveau Nom = ' + i.Nom + ', Nouveau Pays = ' + i.Pays, GETDATE()
    FROM inserted AS i;--ici avec instead of  on ajoute explicitement "AS i" comme alias sinon il va pas reconnaitre la table virtuelle inserted
END;
```

0 %

Messages

• Syntaxe générale d un déclencheur en SQL SERVER

On va travailler directement avec un exemple concret

- **Déclencheurs INSTEAD OF: INSTEAD OF UPDATE**
- **Exemple**

Effectuer une mise à jour dans la vue Vueclients puis afficher le contenu de la table clients et journal pour voir le travail effectué par le déclencheur automatiquement après la mise à jour dans la vue

```
-- Mettre à jour le pays et le nom du client dans la vue
UPDATE VueClients SET Nom = 'Wided', Pays = 'Tunisie' WHERE Nom = 'Bob';
```

100 %

Messages

```
(1 ligne affectée)

(1 ligne affectée)

(1 ligne affectée)

(1 ligne affectée)

Heure de fin : 2025-11-03T13:24:50.9336866-05:00
```

• Syntaxe générale d un déclencheur en SQL SERVER

On va travailler directement avec un exemple concret

- **Déclencheurs INSTEAD OF: INSTEAD OF UPDATE**
- **Exemple**

Effectuer une mise à jour dans la vue Vueclients puis afficher le contenu de la table clients et journal pour voir le travail effectué par le déclencheur automatiquement après la mise à jour dans la vue

```
--tester le contenu de la table clients aprés execution du déclencheur trg_InsteadOfUpdate_VueClients
SELECT * FROM Clients;
```

100 %

Résultats | Messages

	Id_Client	Nom	Pays
1	2	Wided	Tunisie

la mise à jour a été effectué dans la table clients

INSTEAD OF UPDATE dans la vue

```
--tester le contenu de la table journal aprés execution du déclencheur trg_InsteadOfUpdate_VueClients
SELECT * FROM Journal;
```

100 %

Résultats | Messages

	Id_Journal	Action	DateAction
1	1	Nouveau client ajouté : Alice	2025-11-03 10:31:53.467
2	2	Client mis à jour : Ancien pays = Canada, Nouveau ...	2025-11-03 10:57:24.980
3	3	Client supprimé : Alice	2025-11-03 12:13:00.760
4	4	Nouveau client ajouté : Bob	2025-11-03 12:47:50.970
5	5	Client mis à jour : Ancien pays = Maroc, Nouveau p...	2025-11-03 13:24:50.923
6	6	Mise à jour via la vue : Nouveau Nom = Wided, No...	2025-11-03 13:24:50.923

Les déclencheurs en SQL

- **Syntaxe générale d un déclencheur en SQL SERVER**

On va travailler directement avec un exemple concret

- **Déclencheurs INSTEAD OF: INSTEAD OF UPDATE**
- **Exemple**
- SELECT * FROM VueClients; -- la vue n a pas été mise à jour directement avec UPDATE VueClients SET Nom = 'Wided', Pays = 'Tunisie' WHERE Nom = 'Bob'; mais la vue lit ses données depuis la table Clients et c'est le déclencheur qui a effectivement mis à jour la table Clients. Donc la vue sera mise à jour indirectement

```
SELECT * FROM VueClients; --Ce n'est pas parce que la vue a été mise à jour directement.
-- C'est parce que la vue lit ses données depuis la table Clients,
--et c'est le déclencheur qui a effectivement mis à jour la table Clients.
```

00 %

Résultats | Messages

	Id_Client	Nom	Pays
1	2	Wided	Tunisie

Les déclencheurs en SQL

• Syntaxe générale d un déclencheur en SQL SERVER

On va travailler directement avec un exemple concret

- **Déclencheurs INSTEAD OF: INSTEAD OF DELETE**
- **Exemple**
- On veut créer un déclencheur qui Remplace une tentative de suppression d un client sur la vue par une suppression réelle dans la table Clients et l enregistrement de cette action dans la table journal

```sql
-- Creation d un déclencheur qui remplace la suppression sur la vue
--par une vraie suppression dans Clients et enregistre l'action dans Journal.
CREATE TRIGGER trg_InsteadOfDelete_VueClients
ON VueClients
INSTEAD OF DELETE
AS
BEGIN
    DELETE FROM Clients
    WHERE Id_Client IN (SELECT Id_Client FROM deleted);

    INSERT INTO Journal (Action, DateAction)
    SELECT
        'Suppression via la vue : ' + Nom,
        GETDATE()
    FROM deleted;
END;
```

100 %

Messages

Commandes réussies.

Heure de fin : 2025-11-03T13:35:11.6761579-05:00

Les déclencheurs en SQL

• Syntaxe générale d un déclencheur en SQL SERVER

On va travailler directement avec un exemple concret

- **Déclencheurs INSTEAD OF: INSTEAD OF DELETE**
- **Exemple**

Effectuer une suppression d un client dans la vue Vueclients puis afficher le contenu de la table clients et journal pour voir le travail effectué par le déclencheur automatiquement après la suppression dans la vue

```
--suppression d un client de la vue pour déclencher le déclencheur trg_InsteadOfDelete_VueClients
DELETE FROM VueClients WHERE Nom = 'Wided';
```

100 %

Messages

```
(1 ligne affectée)

(1 ligne affectée)

(1 ligne affectée)

(1 ligne affectée)

Heure de fin : 2025-11-03T13:54:52.5261084-05:00
```

Les déclencheurs en SQL

• Syntaxe générale d un déclencheur en SQL SERVER

On va travailler directement avec un exemple concret

- **Déclencheurs INSTEAD OF: INSTEAD OF DELETE**
- **Exemple**

Effectuer une suppression d un client dans la vue Vueclients puis afficher le contenu de la table clients et journal pour voir le travail effectué par le déclencheur automatiquement après la suppression dans la vue

```
-- tester le contenu de la table client aprés execution du déclencheur trg_InsteadOfDelete_VueClients
SELECT * FROM Clients;
```

100 %

Résultats | Messages

Id_Client	Nom	Pays

```
--tester le contenu de la table journal aprés execution du déclencheur trg_InsteadOfDelete_VueClier
SELECT * FROM Journal;
```

100 %

Résultats | Messages

	Id_Journal	Action	DateAction
1	1	Nouveau client ajouté : Alice	2025-11-03 10:31:53.467
2	2	Client mis à jour : Ancien pays = Canada, Nouveau ...	2025-11-03 10:57:24.980
3	3	Client supprimé : Alice	2025-11-03 12:13:00.760
4	4	Nouveau client ajouté : Bob	2025-11-03 12:47:50.970
5	5	Client mis à jour : Ancien pays = Maroc, Nouveau p...	2025-11-03 13:24:50.923
6	6	Mise à jour via la vue : Nouveau Nom = Wided, No...	2025-11-03 13:24:50.923
7	7	Client supprimé : Wided	2025-11-03 13:54:52.510
8	8	Suppression via la vue : Wided	2025-11-03 13:54:52.510

Les déclencheurs en SQL

- **Syntaxe générale de manipulation de déclencheur en SQL SERVER**
- **Suppression d un déclencheur**

✓ **DROP TRIGGER NomDuDeclencheur;**

- **Désactivation d un déclencheur**

✓ **ENABLE TRIGGER NomDuDeclencheur ON NomDeLaTable;**

- **Réactivation d un déclencheur**

✓ **ENABLE TRIGGER NomDuDeclencheur ON NomDeLaTable;**

- **Désactivation de tous les déclencheurs d une table**

✓ **DISABLE TRIGGER ALL ON NomDeLaTable;**

- **Résactivation de tous les déclencheurs d une table**

✓ **ENABLE TRIGGER ALL ON NomDeLaTable;**

Les déclencheurs en SQL

- **Syntaxe générale d un déclencheur en MYSQL Workbench**

```
CREATE TRIGGER nom_du_declencheur
BEFORE|AFTER|INSTEAD OF
INSERT|UPDATE|DELETE
ON nom_de_table
FOR EACH ROW
BEGIN
    -- Instructions SQL à exécuter
END;
```

- - **moment** indique si le déclencheur s'exécute **avant** ou **après** l'événement.

– l évenement, l action qui active le déclencheur

- - Indique que le déclencheur s'exécute pour chaque ligne affectée par l'opération.

Les déclencheurs en SQL

- **Syntaxe générale d un déclencheur en MYSQL Workbench**

On va travailler directement avec un exemple concret utiliser mysql workbench

- **Déclencheurs BEFORE: BEFORE INSERT**
- **Exemple; créer la base gestionclients1**

```
CREATE DATABASE gestionclients1;
USE gestionclients1;
CREATE TABLE Clients (
 Id_Client INT AUTO_INCREMENT PRIMARY KEY,
Nom VARCHAR(50),
 Pays VARCHAR(50));

CREATE TABLE Journal (
  Id_Journal INT AUTO_INCREMENT PRIMARY KEY,
Action VARCHAR(255),
DateAction DATETIME);
```

Les déclencheurs en SQL

• Syntaxe générale d un déclencheur en MYSQL Workbench

On va travailler directement avec un exemple concret utiliser mysql workbench

- **Déclencheurs BEFORE: BEFORE INSERT**

```
CREATE TRIGGER trg_BeforeInsert_Client
BEFORE INSERT ON Clients  --BEFORE INSERT : le trigger s'exécute avant l'insertion.
FOR EACH ROW
BEGIN
  INSERT INTO Journal (Action, DateAction)
  VALUES (CONCAT('Ajout du client : ', NEW.Nom), NOW());
END;
```

- NEW.Nom : fait référence à la **valeur du champ Nom** du nouveau client. NEW C est une table virtuelle comme inserted en sqlserver et OLD comme deleted en sqlserver
- NOW () c est comme GETDATE() en sqlserver

➢ On ajoute une entrée dans Journal avant même que le client soit inséré dans Clients.

Les déclencheurs en SQL

• Syntaxe générale d un déclencheur en MYSQL Workbench

On va travailler directement avec un exemple concret utiliser mysql workbench

- **Déclencheurs BEFORE: BEFORE UPDATE**

Il s'exécute avant la modification d'un client.

```
CREATE TRIGGER trg_BeforeUpdate_Client
BEFORE UPDATE ON Clients
FOR EACH ROW
BEGIN
   INSERT INTO Journal (Action, DateAction)
   VALUES (CONCAT('Avant modification : Ancien nom = ', OLD.Nom, ', Nouveau nom = ',
NEW.Nom), NOW());
END;
```

- OLD.Nom = ancienne valeur du champ. NEW.Nom = nouvelle valeur.

➢ Le journal garde une trace avant la mise à jour.

Les déclencheurs en SQL

• Syntaxe générale d un déclencheur en MYSQL Workbench

On va travailler directement avec un exemple concret utiliser mysql workbench

- **Déclencheurs BEFORE: BEFORE DELETE**

Il s'exécute avant la suppression d'un client.

```
CREATE TRIGGER trg_BeforeDelete_Client
BEFORE DELETE ON Clients
FOR EACH ROW
BEGIN
   INSERT INTO Journal (Action, DateAction)
   VALUES (CONCAT('Suppression du client : ', OLD.Nom), NOW());
END;
```

- OLD.Nom représente la ligne qui va être supprimée.

➢ On garde une trace avant suppression.

Les déclencheurs en SQL

- **Syntaxe générale d un déclencheur en MYSQL Workbench**

On va travailler directement avec un exemple concret utiliser mysql workbench

- **Déclencheurs BEFORE: BEFORE INSERT |BEFORE UPDATE| BEFORE DELETE**

```
-- Insertion
INSERT INTO Clients (Nom, Pays) VALUES ('Alice', 'France');

-- Mise à jour
UPDATE Clients SET Nom = 'Alice Dupont' WHERE Id_Client = 1;

-- Suppression
DELETE FROM Clients WHERE Id_Client = 1;

-- Voir les journaux
SELECT * FROM Journal;
```